MoA Chain: An Architecture for a Blockchain Protocol Supporting a Decentralised Mixture of Agents

Daniel Radu
Dept. of Computers
Gheorghe Asachi Technical University
Iași, România
daniel.radu@academic.tuiasi.ro

Alexandru Archip
Dept. of Computers
Gheorghe Asachi Technical University
Iași, România
alexandru.archip@academic.tuiasi.ro

Silviu-Dumitru Pavăl
Dept. of Computers
Gheorghe Asachi Technical University
Iași, România
silviu-dumitru.paval@academic.tuiasi.ro

***Abstract*—This paper proposes MoA Chain, a blockchain protocol designed to coordinate multiple coding experts in a decentralized, auditable, and scalable inference network. The goal of the protocol is to produce a better canonical answer for the user by decentralizing the conventional Mixture-of-Agents (MoA) approach. In the proposed architecture, user prompts are interpreted as transactions. Each transaction is prioritized by the revenue it is expected to produce in the network and labelled with coding subdomains defined by the protocol. A typical blockchain round is split into mini-rounds. The first mini-round selects, in a verifiable way, the prompt batch and reaches consensus on its coding subdomains; the second selects domain-specific experts and executes the prompts; the third aggregates the candidate answers into a canonical response and supports reward or penalty updates. We implement and evaluate the first mini-round and show that a strict leader-label rule fails on secondary-label disagreement while a refined vote-based rule reaches consensus; the second and third mini-rounds are presented at the design level.**

***Index Terms*—blockchain, mixture-of-agents, LLM, decentralization, consensus, Byzantine fault tolerance.**

I. Introduction

Modern large language model (LLM) inference is increasingly delivered through centralized services, where prompt routing, model execution, answer comparison, and final aggregation are usually hidden from the user. This limits auditability and makes it difficult to understand how a response was produced, whether alternative specialized models could have generated a better answer, or whether the final output was affected by hallucinations, prompt injection, or intermediate-context manipulation. While single-agent pipelines are simple to deploy, they concentrate both decision-making and failure modes in one inference path.

Mixture-of-Agents (MoA) architectures address part of this limitation by combining the outputs of multiple models or agents in order to improve response quality [1]. However, most existing MoA systems still rely on centralized orchestration. In such settings, the entity that selects the agents, routes the prompt, compares the answers, and produces the final output remains a trusted coordinator. As a result, current MoA approaches do not provide a consensus-level mechanism for selecting specialized agents, validating semantic agreement among their outputs, rewarding or penalizing participating nodes, or tracing the complete lifecycle of a prompt.

This paper proposes *MoA Chain*, a blockchain-based protocol design for decentralized, collaborative, and traceable AI inference. The main challenge addressed by the protocol is the combination of two fundamentally different execution models: the deterministic validation model of blockchain systems and the probabilistic behavior of AI agents. Traditional blockchain consensus is usually defined over deterministic state transitions, such as balance updates, nonces, and block hashes. In contrast, AI inference may produce semantically similar but syntactically different outputs across agents. Therefore, a decentralized AI inference protocol cannot rely only on conventional block validation; it must also introduce additional consensus steps for prompt classification, expert selection, answer comparison, semantic agreement, and canonical response generation.

The design of MoA Chain is guided by five main goals: representing prompts as traceable transactions, routing them to specialized agents, enabling collaborative answer generation, rewarding useful participants while penalizing faulty or malicious behavior, and producing canonical answers that may later support model improvement.

The main contributions of this paper are:

1) We propose *MoA Chain*, a blockchain-based protocol design for decentralized Mixture-of-Agents inference.
2) We introduce a multi-step consensus flow that separates prompt selection, semantic labeling, expert routing, answer generation, aggregation, and canonical response validation.
3) We define a subdomain-based mechanism for selecting specialized agents, reducing the influence of a single centralized orchestrator over the inference process.
4) We outline a design for integrating semantic agreement, rewards, penalties, and traceability into the consensus layer in order to support accountable AI inference, and we implement and evaluate the first mini-round.

II. Related Work

Recent research has explored several directions for improving the quality, trustworthiness, and robustness of Large Language Model (LLM) systems. The most relevant directions for MoA Chain are multi-agent LLM inference, blockchain-assisted LLM coordination, decentralized expert networks, and trustless inference verification. These works provide important building blocks, but they do not fully combine prompt-as-transaction processing, fee-based prioritization, quorum-backed semantic routing, expert answer generation, canonical aggregation, and incentive updates within a single blockchain protocol.

A. Multi-Agent LLM Inference

Wang et al. demonstrated that multiple LLMs can collaborate in order to generate responses of higher quality than those produced by a single model through the Mixture-of-Agents (MoA) architecture [1]. In their approach, several models first generate candidate responses, while an aggregator model synthesizes these outputs into a final answer. This result is directly relevant to MoA Chain because it supports the idea that answer quality can be improved through collaboration and aggregation rather than through a single inference path. However, MoA remains based on centralized orchestration. The system still relies on a trusted coordinator that selects the models, routes the prompt, collects the candidate answers, and produces the final output.

B. Blockchain-Based Multi-Agent Coordination

Luo et al. introduced a Weighted Byzantine Fault Tolerance consensus mechanism for a trusted multi-LLM network [2]. Their framework allows multiple LLMs to participate in answering a user request, while a blockchain-driven consensus mechanism determines whether a response should be accepted. A central idea of their approach is that models should not have equal influence in consensus, but should instead be weighted according to response quality and trustworthiness. Nevertheless, the mechanism is closer to leader-response validation than to collaborative response synthesis. The leader generates a response, followers compare their own outputs against the leader's answer, and consensus is used to determine whether that answer should be accepted.

AgentChain proposes a blockchain-empowered multi-agent framework for trustworthy LLM question-answering systems [3]. It replaces centralized orchestration with a decentralized council and introduces Proof-of-Content-Quality (PoCQ), where selected workers generate candidate answers and miners evaluate and vote on the final response. AgentChain also introduces stake-based incentives and penalties to reduce the influence of malicious agents. This makes it one of the closest works to MoA Chain from the perspective of semantic consensus and adversarial robustness. However, AgentChain focuses on question-answering robustness and treats the blockchain mainly as an abstract ledger for coordination and accountability. Moreover, its final answer is obtained through miner evaluation and voting, rather than through a protocol-level canonical aggregation step derived from a majority cluster of semantically similar expert answers.

Jo and Park proposed DecentLLMs, a leaderless Byzantine-robust coordination mechanism for LLM agents [4]. In this design, worker agents generate answers in parallel, while evaluator agents independently score and rank the answers. The final output is selected as the answer with the highest robustly aggregated score, computed using the geometric median over evaluator score vectors. This approach improves over leader-based protocols because it evaluates multiple candidate answers in the same round and avoids finalizing an underperforming leader response. However, DecentLLMs aggregates evaluations rather than answer contents: the final answer is selected from the existing candidates, not synthesized as a canonical response from multiple semantically consistent outputs.

C. Decentralized Expert and Inference Networks

LLM-Net proposes a blockchain-based LLM-as-a-Service framework in which coordinators route user queries to specialized LLM providers according to declared expertise and historical performance records stored on-chain [5]. The selected respondents collaborate through multi-agent debate and produce a final consolidated answer, while smart contracts define participation terms and reward distribution. This work is close to MoA Chain at the architectural level because it combines expert selection, collaborative reasoning, blockchain records, and incentives. However, its per-query workflow remains coordinator-driven: the coordinator selects respondents, orchestrates the reasoning process, verifies the final answer, and delivers it to the requester. Therefore, LLM-Net decentralizes the service marketplace and audit layer, but does not define a validator-governed blockchain protocol for semantic routing, answer aggregation, and finalization.

PolyLink proposes a blockchain-based decentralized edge AI platform for LLM inference [6]. It routes inference requests to edge workers, supports single-device and cross-device model execution, and introduces the Trustless Inference Quality Evaluation (TIQE) protocol, where validators assess worker outputs using Cross-Encoder and LLM-as-a-Judge mechanisms. Its blockchain layer is used for validator committee selection, model-quality score consensus, slashing, and reward distribution. PolyLink is relevant because it addresses trustless LLM inference and incentive alignment in a decentralized setting. However, it focuses on verifying the quality of inference performed by workers, rather than coordinating multiple expert agents to generate, compare, aggregate, and finalize a canonical answer for each prompt.

Ogunsina proposed a Hashgraph-inspired consensus mechanism for reliable multi-model reasoning [7]. Their work studies how several models can exchange outputs through a Hashgraph-inspired communication process and iteratively converge toward a more reliable final answer. This is relevant to MoA Chain because it addresses semantic agreement over non-deterministic model outputs, where different answers may be syntactically different but semantically similar. However,

the approach is mainly a consensus-inspired multi-model deliberation mechanism and does not define an end-to-end blockchain protocol for prompt transactions, economic validation, expert routing, and answer finalization.

D. Comparative Summary

Table I summarizes the main differences between MoA Chain and the related approaches. The closest works provide either centralized MoA aggregation, blockchain-assisted answer validation, robust answer selection, coordinator-driven expert routing, or trustless inference evaluation. MoA Chain differs by treating user prompts as native blockchain transactions and by integrating transaction prioritization, quorum-backed semantic labeling, domain-aware expert selection, replicated inference, canonical answer aggregation, and reward or penalty updates into the protocol workflow itself.

III. MoA Chain

MoA Chain is designed as a blockchain-based protocol for decentralized Mixture-of-Agents inference. Its objective is to transform user prompts into traceable protocol objects, route them to specialized agents, aggregate the generated answers, and validate the resulting canonical response through consensus. In this section, we describe the main components of the protocol and the design decisions required to adapt a blockchain execution model to AI inference.

A. Blockchain Baseline and Impediments

MoA Chain assumes a validator-based blockchain model in which each round elects a consensus group composed of one leader and multiple validators. The leader proposes a block, while the validators independently verify the block header and body, including nonce continuity, transaction ordering, balance constraints, block limits, and the consistency of the computed block hash. If a validator accepts the proposal, it signs the block hash and sends the signature to the leader. A block is committed when the leader collects a quorum of at least $2f+1$ valid signatures and broadcasts the resulting certificate to the network [8].

This baseline model is appropriate for deterministic state transitions, where all honest validators can independently compute the same result from the same input. However, MoA Chain extends this model because the protocol does not only validate balance updates, nonces, or block hashes. It also needs to validate prompt selection, prompt classification, expert routing, answer comparison, semantic agreement, and canonical response generation. These operations involve AI agents and may produce outputs that are semantically similar but syntactically different. Therefore, the protocol must introduce additional coordination steps in order to make non-deterministic inference compatible with blockchain-style consensus. We assume that validators may run heterogeneous models, which is what makes the mixture useful.

The first design challenge concerns transaction selection. The protocol must define a deterministic approximation of prompt consumption, prioritizing transaction by revenue and preserving the intuition of fee and consumption based transaction selection while adapting it to prompt based execution.

The second design challenge concerns the influence of the leader. As in many blockchain protocols, the leader proposes a batch of selected transactions. In MoA Chain, however, the selected prompts must also be labeled with subdomains, and these subdomains are later used to select the specialized validators that execute the prompts. The frequency of each subdomain in the selected batch influences the probability of selecting validators specialized in that domain. Consequently, a malicious leader could try to manipulate the subdomain distribution in order to influence the next expert group. To reduce this attack surface, the protocol must validate not only the selected transactions, but also the semantic labels and the resulting subdomain frequency map.

The third design challenge concerns the object over which consensus is reached. In a traditional blockchain round, validators sign a hash of the validated block. Any modification to the block changes the hash and invalidates the corresponding signature. This mechanism works well when the validated data is deterministic. In MoA Chain, however, prompt labels and generated answers may differ across validators even when they are semantically compatible. If such non-deterministic outputs are included directly in the signed object without additional agreement rules, honest validators may compute different hashes and fail to contribute to the same quorum. For this reason, MoA Chain separates deterministic block validation from semantic agreement and introduces mini-rounds that progressively validate transaction selection, subdomain agreement, expert execution, and canonical answer generation.

B. Mini-Round Based Consensus Design

Starting from the baseline blockchain model previously described, MoA Chain extends the conventional consensus round by introducing a mini-round based execution flow. The main objective of this design is to select a suitable group of expert validators and enable them to generate a better, canonical, and traceable answer for each user prompt. A single blockchain round is not sufficient for this objective, because the protocol must validate not only deterministic state transitions, but also prompt selection, semantic labeling, expert routing, answer generation, and final response aggregation. For this reason, MoA Chain divides a complete protocol round into three mini-rounds.

The first mini-round is responsible for selecting the prompt transactions that will be executed later and for establishing the canonical subdomain distribution of the selected batch. Validators verify the block structure, the transaction validity rules, and the proposed semantic labels.

After consensus is reached on the selected prompts and their subdomain distribution, the second mini-round starts. In this stage, the validator selection algorithm is influenced by the canonical subdomain frequencies computed in the first mini-round and by the domain-specific scores of the validators. As a result, validators with higher scores in the relevant subdomains have a higher probability of being selected as experts for the

TABLE I
COMPARISON BETWEEN MOA CHAIN AND RELATED APPROACHES.

Approach	Multi-agent	Blockchain	Decentralization	Prompt tx. / priority	Domain routing	Canonical answer	Incentives	Model updates	Malicious attack prevention
MoA [1]	Yes	No	Centralized coordinator	No	No	Aggregated by model	No	No	No
WBFT Multi-LLM [2]	Yes	Yes	Leader-based	No	No	Leader response	Yes	No	Byzantine leader/LLM tolerance
AgentChain [3]	Yes	Abstract ledger	Council-based	No	No	Miner proposal / resolution	Yes	No	Poisoning, backdoor, jailbreak
DecentLLMs [4]	Yes	On-chain records	Leaderless evaluators	Request only	No	Best scored answer	Limited	No	Byzantine workers/evaluators
LLM-Net [5]	Yes	Smart contracts / records	Coordinator-driven	Query contract	Yes, by coordinator	Debate-consolidated	Yes	Indirect via reputation	Reputation-based; no formal BFT threat model
PolyLink [6]	No	Yes	Decentralized edge workers	Inference request / pricing	Model routing only	Single inference output	Yes	No	Model degradation, validator corruption
Hashgraph reasoning [7]	Yes	Consensus-inspired	Peer deliberation	No	No	Convergent answer	No	No	Faulty model tolerance
MoA Chain	Yes	Protocol-level	Validator-governed	Yes, mempool + fee/tip priority	Quorum-backed semantic routing	Aggregated canonical answer	Yes	Future canonical-data use	Byzantine / semantic manipulation

TABLE II
MAIN FIELDS OF A PROMPT TRANSACTION.

Field	Purpose
`sender`	Wallet address that creates the prompt transaction.
`signature`	Sender authorization over the transaction hash.
`nonce`	Deterministic ordering of transactions per sender.
`prompt`	User request to be executed by AI agents.
`promptTokens`	Token count extracted from the prompt.
`outputTokenLimit`	User-provided upper bound for the answer size.
`thinkingModeTokens`	Protocol-defined budget for the selected thinking mode.
`estimatedConsumption`	Estimated inference cost of the transaction.
`estimatedFee`	Estimated protocol fee paid by the sender.
`tip`	Additional incentive used for prioritization.
`txHash`	Unique transaction identifier.
`labels`	Coding subdomains associated with the prompt.

current batch. Each node deterministically derives the same consensus group and the same leader for this mini-round.

The purpose of the second mini-round is to execute the selected prompt transactions and collect candidate answers from the selected expert validators. The leader executes the transactions and proposes the resulting executed block, while the validators independently validate the block and generate their own answers for the same transactions. The leader can form an aggregation set only when at least $2f+1$ answers are considered semantically similar. This majority group of similar answers becomes the basis for generating the canonical answer in the following mini-round.

The third mini-round is responsible for producing the final canonical answer for each transaction and for identifying answers that behave as outliers. We assume that this round also has a different consensus group. This step is essential not only for answer generation, but also for accountability, because the protocol must determine which validators contributed useful outputs and which validators produced faulty, malicious, or clearly divergent answers. These decisions are later used for reward and penalty updates. In this stage, semantic similarity between answers can be evaluated using deterministic embedding-based similarity and clustering techniques, such as sentence embeddings and density-based clustering [9], [10].

C. Prompt Transactions

In MoA Chain, each user prompt is represented as a transaction. This allows prompts to be ordered, validated, selected, executed, traced, and charged through blockchain-style mechanisms. The main fields of a prompt transaction are summarized in Table II.

The transaction hash is signed by the sender using a digital signature scheme such as Ed25519 [11]. This associates each prompt with an accountable wallet and allows the protocol to charge the sender for the execution cost and the attached tip. This economic cost also discourages repeated abusive or malicious prompt submissions, since users must spend resources for each attempt. In the current proof-of-concept, MoA Chain focuses on coding-related prompts, and the `labels` field is restricted to a fixed set of coding subdomains used later for expert selection.

The estimated consumption of a transaction combines the number of prompt tokens, the expected output size, and the token budget associated with the selected thinking mode:

$$\widehat{C}(tx) = T_p(tx) + T_o(tx) + T_m(tx), \tag{1}$$

where $T_p(tx)$ is computed using the `cl100k_base` tokenizer, $T_o(tx)$ is the user-provided output token limit, and $T_m(tx)$ is the protocol-defined thinking-mode budget [12], [13]. The estimated fee is then computed as:

$$\widehat{F}(tx) = F_b + p_u \cdot \widehat{C}(tx), \tag{2}$$

where F_b is the base fee and p_u is the unit price per estimated token. A transaction is economically valid only if the sender can cover both the estimated fee and the tip:

$$balance(sender) \geq \widehat{F}(tx) + tip(tx). \tag{3}$$

Finally, the priority of a transaction is defined as:

$$priority(tx) = \frac{tip(tx)}{\widehat{C}(tx)} = \frac{tip(tx)}{T_p(tx) + T_o(tx) + T_m(tx)}. \tag{4}$$

This follows the intuition of blockchain transaction prioritization, where higher incentives increase inclusion probability, while resource consumption limits the priority of expensive transactions [14]–[16].

D. Mempool

The mempool stores pending prompt transactions before block inclusion. It uses two indexes: a transaction map indexed by `txHash`, and a sender map indexed by `sender`. The sender map stores each sender's transactions ordered by nonce. When two transactions have the same nonce, the one with lower estimated consumption is ordered first; if the estimated consumption is also equal, the transaction hash is used as a deterministic tie-breaker.

This structure allows the selection algorithm to consider only the first executable transaction of each sender, while preserving nonce continuity and deterministic ordering across nodes.

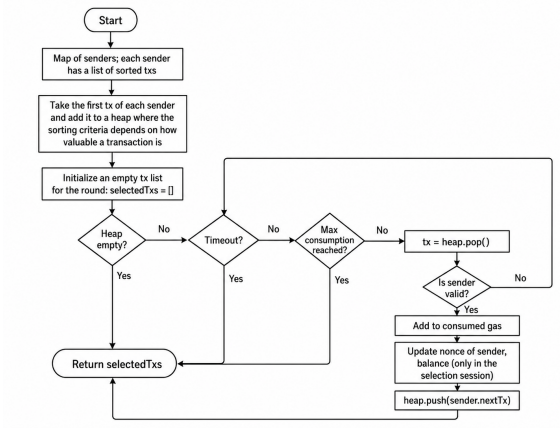

Fig. 1. Heap-based prompt transaction selection with nonce continuity, validity checks, and block-consumption limits.

E. Transaction Selection Algorithm

The selection algorithm constructs the batch of prompt transactions proposed in the first mini-round. It must be deterministic, efficient, bounded by time or block-consumption limits, and consistent with the protocol validity rules, including nonce continuity and sufficient sender balance.

At the beginning of selection, the node creates a concurrent-safe snapshot of the mempool and inserts into a heap only the first pending transaction of each sender. The heap orders transactions by descending priority. Ties are resolved by lower estimated consumption and then by transaction hash.

During the selection loop, the algorithm repeatedly pops the highest-priority transaction from the heap. If the transaction is valid and does not exceed the remaining block consumption limit, it is added to the selected batch and the temporary account state is updated. Then, the next transaction from the same sender becomes eligible and is inserted into the heap. The loop stops when the heap is empty, the block consumption limit is reached, or the selection timeout expires. Figure 1 illustrates this process.

F. Mini-Round One: Semantic Label Agreement

The first mini-round is responsible for selecting the prompt transactions that will be executed in the following mini-round and for reaching agreement on their coding subdomains. A consensus group is selected deterministically, using the global score of each validator. The first selected validator acts as the leader, while the remaining selected validators participate in block validation and voting.

The leader creates a proposed block by linking it to the previous chain state through the previous root hash, previous block hash, round, mini-round, and nonce. It then runs the transaction selection algorithm described above in order to construct the block body.

In the initial proof-of-concept, the leader also labelled each selected prompt with a set of coding subdomains sorted by dominance and computed the corresponding subdomain frequency map. More specifically, the leader generated N labels for each transaction but proposed only the first $N/2$ labels. Each validator independently validated the block header, transaction ordering, economic validity, nonce continuity, and block consumption limits. In addition, each validator generated its own N labels for every selected transaction and accepted the leader's labels only if all proposed labels were included in its locally generated label set.

Although this design was simple, it made semantic agreement dependent on the leader's proposed labels. Therefore, the refined design moves semantic label evidence into validator votes. In this version, each validator independently generates a transaction-to-label map for the selected block, where each entry associates a transaction hash with the labels assigned by that validator. The validator signs both the block hash and its own semantic label map, so that the vote carries evidence for deterministic block agreement as well as semantic label participation.

After receiving votes, the leader aggregates the signed label maps from a quorum of at least $2f + 1$ validators and derives the finalized subdomain frequency map from this quorum. Each validator receives this certificate and commits the artifact only if the certificate is valid, meaning that the block signatures and semantic-label signatures are correct and the quorum threshold is satisfied. The committed artifact therefore consists of the selected block together with the aggregated subdomain frequencies derived from validator-provided labels, rather than from labels proposed only by the leader. This frequency map is later used by the second mini-round to perform domain-aware expert validator selection. For each transaction hash, only subdomain labels supported by at least the same 2f+1 quorum used for block finalization are retained, so that the finalized frequency map reflects a quorum-backed semantic interpretation rather than a single proposer's view.

This redesign changes the role of the first mini-round from validating a single leader-proposed semantic interpretation to aggregating multiple independent semantic interpretations produced by validators. As a result, the protocol reduces the leader's influence over subdomain assignment and creates a stronger basis for selecting the expert group used in the next mini-round.

G. Validator Scoring and Selection

MoA Chain uses validator scores to influence the probability of being selected in a consensus group. Each validator v has a global score $G(v)$, used for general consensus participation, and a set of domain-specific scores $DScore(v, d)$, used when the protocol needs validators specialized in a particular subdomain d.

The first mini-round uses only the global validator score:

$$S_1(v) = G(v). \tag{5}$$

The second mini-round uses the canonical subdomain distribution computed in the first mini-round. Let $\mathcal{D}$ be the set of subdomains present in the selected batch and let $freq(d)$ be the frequency of subdomain d. The normalized weight of each subdomain is:

$$w_d = \frac{freq(d)}{\sum_{d' \in \mathcal{D}} freq(d')}. \tag{6}$$

The selection score of validator v for the second mini-round is then computed as the weighted sum of its domain-specific scores:

$$S_2(v) = \sum_{d \in \mathcal{D}} w_d \cdot DScore(v, d). \tag{7}$$

Both mini-rounds use the same deterministic weighted selection procedure, but with different scoring functions. For mini-round one, the procedure uses $S_1(v)$; for mini-round two, it uses $S_2(v)$. Each validator receives a selection weight proportional to its score:

$$W(v) = \lfloor \alpha \cdot S(v) \rfloor, \tag{8}$$

where α is a scaling factor. The protocol builds an expanded list containing $W(v)$ occurrences of each validator and samples from it using a deterministic pseudo-random generator initialized with the common round seed. After a validator is selected, all its occurrences are removed from the expanded list, which ensures sampling without replacement. The first selected validator becomes the leader, while the remaining selected validators form the consensus group. Validators whose scaled score floors to zero receive no occurrences and are therefore not eligible in that mini-round, which also discourages negligible-stake spam. This also highlights the importance of penalizing and jailing malicious or faulty validators: repeated misbehavior reduces their future eligibility, and in the worst case their staked value can be withheld or slashed.

This mechanism follows the general intuition of Proof-of-Stake-based validator selection: the first mini-round favors validators with high global reliability, while the second mini-round favors validators specialized in the subdomains present in the selected prompt batch [17].

H. Adversarial Selection Risk

The validator-selection mechanism of MoA Chain is score-weighted. Therefore, the relevant adversarial quantity is not only the number of malicious validators, but also the total selection weight controlled by them. Let $\mathcal{V}$ be the set of eligible validators and let $\mathcal{M} \subseteq \mathcal{V}$ be the subset of malicious validators. Each validator v has a selection weight:

$$W(v) = \lfloor \alpha \cdot S(v) \rfloor, \tag{9}$$

where $S(v)$ is the score used in the current mini-round and α is a scaling factor. For the first mini-round, $S(v)$ corresponds to the global validator score. For the second mini-round, $S(v)$ corresponds to the domain-aware score computed from the finalized subdomain distribution.

1) Probability of Selecting a Malicious Leader: Since the leader is the first validator sampled from the weighted selection list, the probability that the selected leader is malicious is equal to the fraction of total selection weight controlled by malicious validators:

$$P(L \in \mathcal{M}) = \frac{\sum_{v \in \mathcal{M}} W(v)}{\sum_{u \in \mathcal{V}} W(u)}. \tag{10}$$

This probability measures the risk of assigning proposal power to a malicious validator. However, selecting a malicious leader does not imply that the round is corrupted. The leader can propose a block, but finalization still requires a quorum of valid validator signatures. Moreover, in the refined vote-based semantic design, the leader does not unilaterally determine the labels of the selected transactions. Instead, semantic evidence is derived from independently generated and signed validator label maps.

2) Probability of Selecting a Malicious Semantic Quorum: In MoA Chain, the first mini-round finalizes not only a block of selected prompt transactions, but also the semantic evidence used to derive the subdomain-frequency map for the next expert-selection step. Therefore, a relevant adversarial event is the selection of a malicious semantic quorum, i.e., enough colluding validators to influence or finalize an incorrect semantic certificate.

As a baseline, we first consider the equal-weight case, where all eligible validators have the same selection weight and the consensus group is sampled without replacement. Let N be the number of eligible validators, M the number of malicious validators, and c the size of the selected consensus group. The protocol tolerates:

$$f = \left\lfloor \frac{c-1}{3} \right\rfloor \tag{11}$$

malicious validators inside the selected group, and the quorum threshold is:

$$q = 2f + 1. \tag{12}$$

In this equal-weight baseline, the number of malicious validators X in the selected consensus group follows a hypergeometric distribution:

$$P(X = k) = \frac{\binom{M}{k}\binom{N-M}{c-k}}{\binom{N}{c}}. \tag{13}$$

Therefore, the probability of selecting at least a malicious semantic quorum is:

$$P(X \geq q) = \sum_{k=q}^{\min(M,c)} \frac{\binom{M}{k}\binom{N-M}{c-k}}{\binom{N}{c}}. \tag{14}$$

This probability is computed for a single mini-round. Since the protocol executes repeatedly, the cumulative probability of observing at least one selected malicious semantic quorum after R rounds is:

TABLE III
BASELINE PROBABILITY OF SELECTING A MALICIOUS SEMANTIC QUORUM UNDER EQUAL VALIDATOR WEIGHTS, WITH $N = 100$ ELIGIBLE VALIDATORS AND $M = 33$ MALICIOUS VALIDATORS.

Consensus group size c	f	Quorum $q = 2f + 1$	One round	500 rounds
10	3	7	1.375%	99.902%
13	4	9	0.484%	91.180%
16	5	11	0.161%	55.380%
20	6	13	0.110%	42.406%
25	8	17	0.00355%	1.757%
31	10	21	0.000146%	0.0730%
40	13	27	$2.15 \cdot 10^{-7}$%	$1.08 \cdot 10^{-4}$%
50	16	33	$3.34 \cdot 10^{-12}$%	$1.67 \cdot 10^{-9}$%

$$P_{\geq 1}(R) = 1 - (1 - P(X \geq q))^R. \tag{15}$$

This estimate assumes independent rounds; in practice the eligible validator set and the weights evolve over time, so it should be read as a first-order approximation rather than an exact bound.

Table III illustrates the baseline equal-weight case for $N = 100$ eligible validators, $M = 33$ malicious validators, and different consensus group sizes. The results are not specific to a single implementation of MoA Chain, but they show the security effect of the selected committee size. Small committees may have a non-negligible probability of selecting a malicious semantic quorum over many rounds, while larger committees significantly reduce the cumulative risk. In MoA Chain, this trade-off is especially relevant because the selected group does not only validate deterministic block data, but also contributes signed semantic label maps used for expert routing in the next mini-round.

IV. PROTOTYPE EVALUATION

The current prototype focuses on validating the first mini-round. The implemented components include the mempool, transaction processor, block proposer, block validator, body executor, validator registry, peer registry, signer, broadcaster, mini-round-one handler, round state, round handler, and round loop. Components such as account state, blockchain state, and the labeler were mocked where full implementations were not required for the tested consensus path.

Three integration scenarios were evaluated, as summarized in Table IV. The first two scenarios validate the normal consensus path: an empty block and a block with intentionally compatible labels are both committed successfully. The third scenario uses 10 validators and 10 prompt transactions, where each validator labels the same transactions using a different agent-generated label file. Each agent was required to assign exactly six labels from the fixed coding-subdomain set. Both validation rules (the strict leader-label rule and the refined vote-based rule) are implemented in the prototype, and Table IV reports the observed outcome under each rule.

In the third scenario, the strict leader-label validation rule did not reach consensus. The quorum threshold was 7 signatures out of 10 validators, but no possible leader obtained enough signatures; the best cases reached only 5 valid signatures. However, all ten agents agreed on the dominant label for every transaction in the batch. Thus, the empirical dominant-label agreement was:

TABLE IV
MINI-ROUND ONE INTEGRATION TEST RESULTS.

Test	Scenario	Strict rule	Vote-based design
T1	Empty block	Commit	Commit
T2	Compatible labels	Commit	Commit
T3	10 txs, 10 agents	Fail	Commit

$$P_{dominant\_agreement} = 1.$$

This shows that requiring *all* leader-proposed labels to be included in each validator's local label set is too restrictive for multi-agent semantic classification.

The most ambiguous transaction was the prompt associated with a React dashboard for cloud infrastructure monitoring. All agents identified `web_front_end` as the dominant label, but diverged on secondary labels such as `cloud_engineering`, `dev_ops`, `databases`, and `mobile_dev`.

The refined collaborative vote-based design resolves this normal-case failure. With this design, the third scenario reaches consensus, and all nodes finalize the same block and the same aggregated subdomain-frequency result.

V. DISCUSSION AND FUTURE WORK

The current prototype validates the feasibility of the first mini-round and shows that a conventional blockchain consensus flow can be extended with semantic metadata required for decentralized AI inference. However, this also changes the security requirements of the protocol: semantic artifacts must be treated with the same rigor as block contents.

A first direction for future work is to harden the vote-based semantic certificate. The transaction-to-label map must be bound exactly to the transactions contained in the proposed block, and the aggregated certificate must validate signer eligibility, quorum size and unique signers. This is necessary because the second mini-round depends on these frequencies for domain-aware expert selection.

A second direction concerns the complete implementation of the second and third mini-rounds. The second mini-round must define how specialized validators execute selected prompts and how a majority group of semantically similar answers is identified. The third mini-round must define how the canonical answer is generated, how outlier answers are detected, and how the network distinguishes between faulty outputs and valid alternative answers that may differ from the majority. A possible approach is to use deterministic answer embeddings and clustering methods for grouping semantically similar responses [9], [10].

A third direction is the reward and penalty mechanism. Future versions must define how validator behavior updates both the global score and the domain-specific scores. This includes rewarding validators whose outputs contribute to the

majority cluster, penalizing faulty or malicious validators, and avoiding unfair penalties for minority answers that are semantically valid. The scoring model may also be extended with nominated staking mechanisms, inspired by Nominated Proof of Stake, where users delegate stake to validators with reliable historical behavior, while both validators and nominators remain exposed to penalties in case of malicious participation [18].

A further concern is inference replication: in the current design every selected validator runs the model on every assigned prompt, so the cost of a round grows with the committee size. The fee model must therefore eventually account for this replication factor rather than for a single inference, and treat the redundancy as the price of decentralized verifiability.

Also, future mini-round implementations should be efficient. The current mini-round decomposition improves separation of responsibilities and reduces the influence of a single leader over the semantic pipeline. However, it also introduces an additional liveness cost. A complete MoA Chain round succeeds only if all mini-rounds complete successfully.

Let p_1, p_2, and p_3 denote the failure probabilities of the first, second, and third mini-round, respectively. Assuming independent failures, the probability that a complete round fails is:

$$P_{fail} = 1 - (1 - p_1)(1 - p_2)(1 - p_3) \tag{16}$$

This shows that even small mini-round failure probabilities accumulate at the round level. Therefore, future implementations must optimize timeout handling, fallback leader selection, retry logic, and message aggregation in order to preserve liveness.

Another important direction concerns transaction admission and resource estimation. Before prompts enter the mempool, the protocol should include a deterministic pre-filtering stage for rejecting malformed, abusive, or policy-violating prompts. The transaction validity rules should also be extended with additional checks on prompt size, output-size limits, and maximum estimated consumption. In parallel, the current token-consumption approximation should be improved through deterministic prompt analysis, keyword-based heuristics, or deterministic classification methods. The protocol should also reconcile the estimated consumption with the actual inference cost after execution, refunding or charging the difference within the committed bounds. This would make transaction prioritization and fee estimation more robust while preserving reproducibility across nodes.

Finally, the protocol must address communication overhead. Later mini-rounds may significantly increase message size because validators exchange answers, labels, frequency maps, signatures, and aggregation artifacts. Future versions should reduce transmitted payloads by using compact commitments, hashes, aggregated signatures, off-chain answer storage, or by transmitting only the metadata required for verification. Another future direction is to use the finalized canonical answers as auditable artifacts for model improvement at the end of an epoch, while carefully preventing poisoning through malicious or low-quality canonical data.

ACKNOWLEDGMENT

The authors used OpenAI's ChatGPT as an AI-assisted tool for language refinement, academic reformulation, structural editing, and preliminary literature discovery. All referenced works were manually checked by the authors before inclusion. The protocol design, implementation, experimental results, analysis, and final editorial decisions remain the responsibility of the authors.

REFERENCES

[1] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=h0ZfDIrj7T

[2] H. Luo, G. Sun, Y. Liu, D. Zhao, D. Niyato, H. Yu, and S. Dustdar, "A weighted byzantine fault tolerance consensus driven trusted multiple large language models network," *IEEE Transactions on Cognitive Communications and Networking*, vol. 12, pp. 3815–3830, 2026.

[3] B. Chen, G. Li, J. Wu, J. Li, M. Chen, and J. Wang, "Agentchain: Blockchain-empowered multi-agent coordination for trustworthy llm question-answering systems," *IEEE Transactions on Dependable and Secure Computing*, 2026, early access / author's version.

[4] Y. Jo and C. Park, "Byzantine-robust decentralized coordination of llm agents," arXiv preprint arXiv:2507.14928, 2025.

[5] Z.-K. Chong, H. Ohsaki, and B. Ng, "Llm-net: Democratizing llms-as-a-service through blockchain-based expert networks," arXiv preprint arXiv:2501.07288, 2025.

[6] H. Liu, J. Cao, B. Yang, D. Bai, Y. Cao, X. Shen, Y. Zhang, J. Liang, S. Jiang, and M. Zhang, "Polylink: A blockchain based decentralized edge ai platform for llm inference," arXiv preprint arXiv:2510.02395, 2025.

[7] K. E. Ogunsina and M. A. Ogunsina, "A hashgraph-inspired consensus mechanism for reliable multi-model reasoning," 2025.

[8] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, 1999, pp. 173–186. [Online]. Available: https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance

[9] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.

[10] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 1996, pp. 226–231.

[11] S. Josefsson and I. Liusvaara, "RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)," https://datatracker.ietf.org/doc/html/rfc8032, 2017.

[12] tiktoken-go, "Tokenizer: Pure go implementation of openai's tiktoken tokenizer," https://github.com/tiktoken-go/tokenizer.

[13] OpenAI, "How to count tokens with tiktoken," https://developers.openai.com/cookbook/examples/how_to_count_tokens_with_tiktoken.

[14] MultiversX, "Gas and fees overview," https://docs.multiversx.com/developers/gas-and-fees/overview/.

[15] Polkadot, "Transactions: Transaction ordering and priority," https://docs.polkadot.com/reference/parachains/blocks-transactions-fees/transactions/.

[16] ——, "Transactions weights and fees," https://docs.polkadot.com/reference/parachains/blocks-transactions-fees/fees/.

[17] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Advances in Cryptology – CRYPTO 2017*, ser. Lecture Notes in Computer Science, vol. 10401. Springer, 2017, pp. 357–388.

[18] Polkadot, "Proof of stake consensus," https://docs.polkadot.com/reference/polkadot-hub/consensus-and-security/pos-consensus/.